

Smart Parking System

using ESP32, RFID, Ultrasonic Sensors & OLED Display

Category: Electronics & Embedded Systems

Platform: ESP32 DevKit V4 | Language: Arduino C++

Simulation: Wokwi Online Simulator

WOKWI LINK : <https://wokwi.com/projects/466798130973985793>

1. Introduction

1.1 Background

Urban traffic congestion and inefficient parking management have become critical challenges in modern cities. Traditional parking systems rely heavily on manual supervision, which leads to delays, human errors, and increased operational costs. An automated smart parking system addresses these issues by leveraging embedded systems technology to provide real-time monitoring, automated access control, and transparent billing.

1.2 Project Aim

To develop an automated smart parking management system that identifies vehicles using RFID cards, controls entry and exit gates automatically, monitors parking slot availability in real time, and calculates parking fees based on parking duration.

1.3 Objectives

- Implement RFID-based vehicle identification for secure and contactless entry/exit.
- Monitor real-time slot occupancy using LED indicators and an OLED display.
- Automate gate control using servo motors triggered by ultrasonic vehicle detection.
- Calculate and display parking duration and billing on an OLED screen.
- Demonstrate the system on a Wokwi simulation platform before physical implementation.

2. System Overview

The Smart Parking System uses an ESP32 microcontroller as the central processing unit. Three parking slots are managed simultaneously. RFID cards are pre-assigned to specific vehicles and slots.

Ultrasonic sensors at the entry and exit gates detect vehicle presence. Servo motors open and close the gates. An SSD1306 OLED display provides a user interface showing slot status, UID, duration, and billing information.

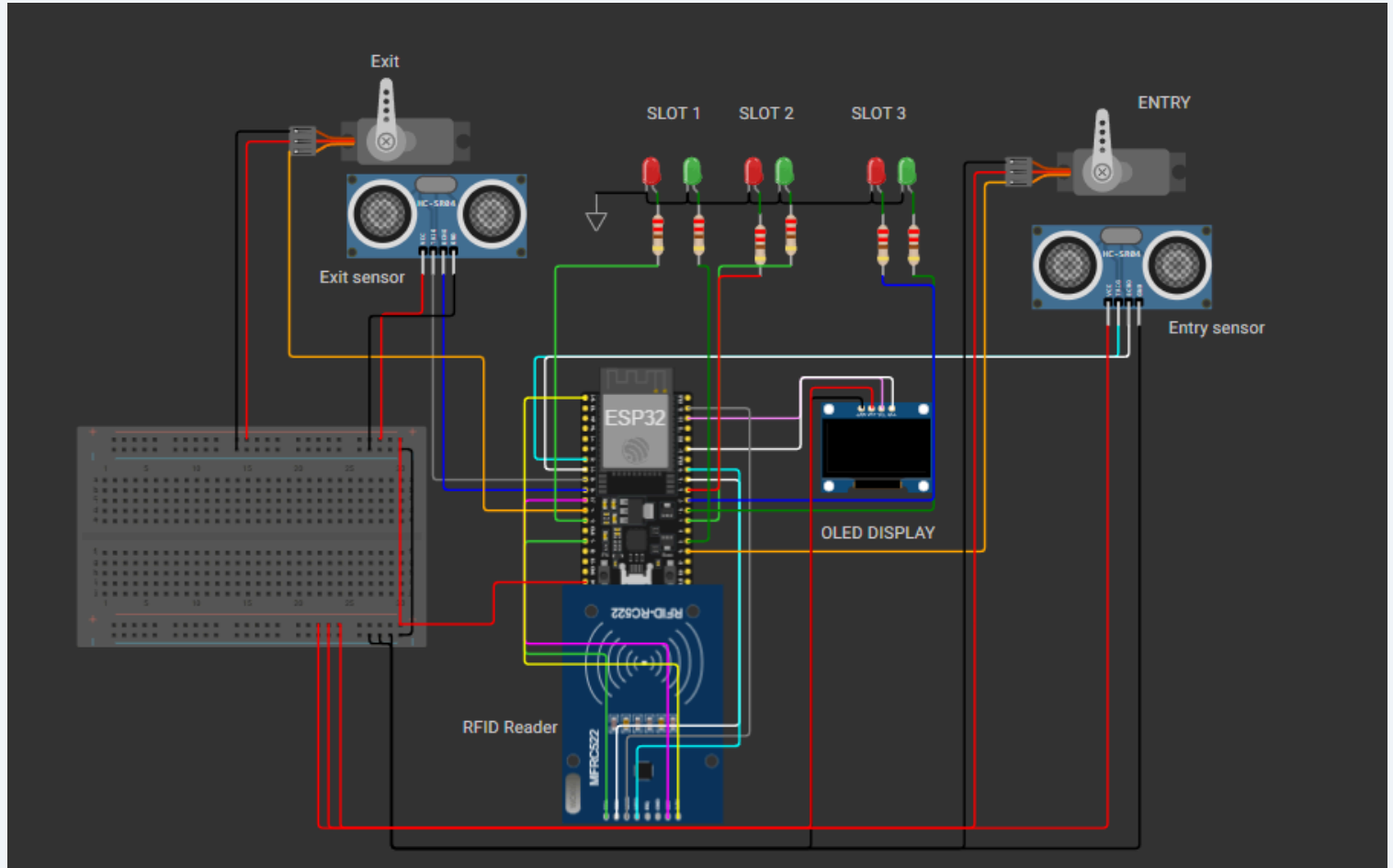


Figure 1: Complete Hardware Setup

2.1 System Block Diagram (Description)

The system architecture flows as follows:

- ESP32 acts as the master controller, interfacing with all peripherals via GPIO, SPI, and I2C buses.
- RFID RC522 module communicates over SPI to read UID from RFID cards.
- OLED SSD1306 communicates over I2C to render real-time status information.
- Two HC-SR04 ultrasonic sensors provide entry and exit vehicle detection signals.
- Two servo motors receive PWM signals from ESP32 to control gate arms.
- Six LEDs (3 green, 3 red) indicate real-time slot availability for each of the three parking bays.

3. Components and Materials

S / N	Component	Specification / Model	Quantity
1	Microcontroller	ESP32 DevKit V4	1
2	RFID Module	RC522 (13.56 MHz, SPI)	1
3	OLED Display	SSD1306, 128×64 px, I2C	1
4	Ultrasonic Sensor	HC-SR04 (2–400 cm range)	2
5	Servo Motor	Standard 5V PWM servo	2
6	Green LED	5 mm, 20 mA	3
7	Red LED	5 mm, 20 mA	3
8	Resistors	220 Ω (current limiting)	6
9	Breadboard	Full-size (830 tie-points)	1
10	Jumper Wires	Male-to-Male / Male-to-Female	As required
11	USB Power Supply	5V / 2A	1

4. Construction and Wiring

4.1 Pin Connection Table

Module	Module Pin	ESP32 GPIO	Notes
OLED SSD1306	SDA	GPIO 21	I2C Data
OLED SSD1306	SCL	GPIO 22	I2C Clock
RFID RC522	SDA (SS)	GPIO 13	SPI Chip Select
RFID RC522	SCK	GPIO 18	SPI Clock
RFID RC522	MOSI	GPIO 23	SPI Master Out
RFID RC522	MISO	GPIO 19	SPI Master In
RFID RC522	RST	GPIO 27	Reset
Entry Ultrasonic	TRIG	GPIO 32	Trigger Pulse
Entry Ultrasonic	ECHO	GPIO 33	Echo Return
Exit Ultrasonic	TRIG	GPIO 25	Trigger Pulse
Exit Ultrasonic	ECHO	GPIO 26	Echo Return
Entry Servo	Signal	GPIO 15	PWM Control
Exit Servo	Signal	GPIO 14	PWM Control

Module	Module Pin	ESP32 GPIO	Notes
Slot 1 – Green LED	Anode (via 220 Ω)	GPIO 2	Available indicator
Slot 1 – Red LED	Anode (via 220 Ω)	GPIO 12	Occupied indicator
Slot 2 – Green LED	Anode (via 220 Ω)	GPIO 4	Available indicator
Slot 2 – Red LED	Anode (via 220 Ω)	GPIO 5	Occupied indicator
Slot 3 – Green LED	Anode (via 220 Ω)	GPIO 16	Available indicator
Slot 3 – Red LED	Anode (via 220 Ω)	GPIO 17	Occupied indicator

4.2 RFID Card-to-Slot Assignment

RFID Card UID	Assigned Slot	Vehicle Indicator
01020304	Slot 1	Green LED GPIO 2 / Red LED GPIO 12
11223344	Slot 2	Green LED GPIO 4 / Red LED GPIO 5
55667788	Slot 3	Green LED GPIO 16 / Red LED GPIO 17

4.3 Wiring Notes

- All LEDs are connected in series with a 220 Ω current-limiting resistor between the GPIO pin and the LED anode; cathodes share a common GND rail.
- RFID RC522 and OLED SSD1306 share the ESP32 3.3V supply rail. Do not power them from 5V.
- Servo motors require a dedicated 5V supply; their signal wires connect to GPIO 14 and GPIO 15.
- HC-SR04 sensors operate at 5V; their ECHO pins output 5V logic. Use a voltage divider (1 k Ω / 2 k Ω) or logic-level shifter before connecting to ESP32 GPIO.

5. Program Flow and Code Structure

5.1 System Flowchart (Text Description)

START → **System Initialization (GPIO, OLED, RFID, Servos)**

↓

OLED displays: 'Smart Parking System – Ready'

All Green LEDs ON | All Red LEDs OFF | Both gates CLOSED

↓

LOOP BEGIN

|

|— Entry Ultrasonic reads distance

| *Is vehicle detected (distance < 15 cm)?*

```

    YES → OLED: 'Scan RFID Card'
        RFID Card Scanned?
        YES → UID Verified in lookup table?
            YES → Slot already occupied?
                NO → Record entry time; set slot OCCUPIED
                    Red LED ON, Green LED OFF
                    OLED: Slot No., UID; Open Entry Gate (90°)
                    Wait 3 sec; Close gate (0°)
                YES → OLED: 'Slot Occupied / Parking Full'
            NO → OLED: 'Unknown Card'

    └ Exit Ultrasonic reads distance
        Is vehicle detected (distance < 15 cm)?
        YES → OLED: 'Scan RFID Card'
            RFID Card Scanned?
            YES → UID verified AND slot marked occupied?
                YES → Calculate duration = exit_time - entry_time
                    Calculate fee (₹ per minute / hour)
                    OLED: Duration, Total Bill
                    Set slot AVAILABLE; Green LED ON, Red LED OFF
                    Open Exit Gate (90°); Wait 3 sec; Close gate (0°)
                NO → OLED: 'No Active Session'

    └ Repeat LOOP
↓
END

```

5.2 Key Functions

Function / Module	Description
measureDistance(trig, echo)	Sends 10 µs pulse on TRIG, measures ECHO pulse duration, returns distance in cm using speed-of-sound formula.
readRFID()	Activates the MFRC522 reader; reads UID bytes from card; formats to HEX string for lookup.
findSlot(uid)	Searches uid-to-slot lookup table; returns slot index or -1 if unknown.
updateLEDs(slot, occupied)	Sets green/red GPIO state for given slot based on occupancy flag.
openGate(servo, angle)	Writes PWM angle (0° = closed, 90° = open) to specified servo object.
handleEntry(uid, slot)	Records millis() entry time; updates slotStatus[]; calls updateLEDs() and openGate().

Function / Module	Description
handleExit(uid, slot)	Computes duration = (millis() - entryTime[slot]) / 60000; calculates bill; displays on OLED; resets slot.
displayOLED(line1, line2...)	Clears SSD1306 buffer; renders multi-line status text at defined coordinates; calls display.display().
calculateBill(minutes)	Returns fee = minutes × ratePerMinute (configurable constant).

6. Source Code (Summarized)

6.1 Library Includes and Global Definitions

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ESP32Servo.h>

// --- Pin Definitions ---
#define SS_PIN 13 // RFID SS
#define RST_PIN 27 // RFID RST
#define ENTRY_TRIG 32 #define ENTRY_ECHO 33
#define EXIT_TRIG 25 #define EXIT_ECHO 26
#define ENTRY_SERVO_PIN 15 #define EXIT_SERVO_PIN 14

int greenPins[] = {2, 4, 16};
int redPins[] = {12, 5, 17};

// --- RFID UID to Slot Mapping ---
String knownUIDs[] = {"01020304", "11223344", "55667788"};
bool slotOccupied[] = {false, false, false};
unsigned long entryTime[] = {0, 0, 0};

// --- Rate: Rs 2 per minute ---
const float RATE_PER_MIN = 2.0;

MFRC522 rfid(SS_PIN, RST_PIN);
Adafruit_SSD1306 display(128, 64, &Wire, -1);
Servo entryServo, exitServo;
```

6.2 Setup Function

```
void setup() {
  Serial.begin(115200);
  SPI.begin();
  rfid.PCD_Init();
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  entryServo.attach(ENTRY_SERVO_PIN);
```

```
exitServo.attach(EXIT_SERVO_PIN);
for (int i = 0; i < 3; i++) {
    pinMode(greenPins[i], OUTPUT);
    pinMode(redPins[i], OUTPUT);
    digitalWrite(greenPins[i], HIGH); // All slots available
    digitalWrite(redPins[i], LOW);
}
entryServo.write(0); exitServo.write(0);
showWelcome();
}
```

6.3 Main Loop

```
void loop() {
    float entryDist = measureDistance(ENTRY_TRIG, ENTRY_ECHO);
    float exitDist  = measureDistance(EXIT_TRIG, EXIT_ECHO);

    if (entryDist < 15) handleEntryFlow();
    if (exitDist  < 15) handleExitFlow();
    delay(200);
}
```

6.4 Distance Measurement

```
float measureDistance(int trig, int echo) {
    digitalWrite(trig, LOW); delayMicroseconds(2);
    digitalWrite(trig, HIGH); delayMicroseconds(10);
    digitalWrite(trig, LOW);
    long dur = pulseIn(echo, HIGH, 30000);
    return (dur * 0.034) / 2;
}
```

6.5 Entry Handling

```
void handleEntryFlow() {
    displayMsg("Vehicle Detected", "Scan RFID Card");
    String uid = waitForRFID();
    int slot = findSlot(uid);
    if (slot == -1) { displayMsg("Unknown Card", ""); return; }
    if (slotOccupied[slot]) { displayMsg("Slot"+String(slot+1), "Occupied!"); return; }
    slotOccupied[slot] = true;
    entryTime[slot] = millis();
    digitalWrite(greenPins[slot], LOW);
    digitalWrite(redPins[slot], HIGH);
    openGate(entryServo, 90);
    displayMsg("Slot "+String(slot+1)+" Assigned", "UID: "+uid);
    delay(3000); openGate(entryServo, 0);
}
```

6.6 Exit Handling and Billing

```
void handleExitFlow() {
```

```

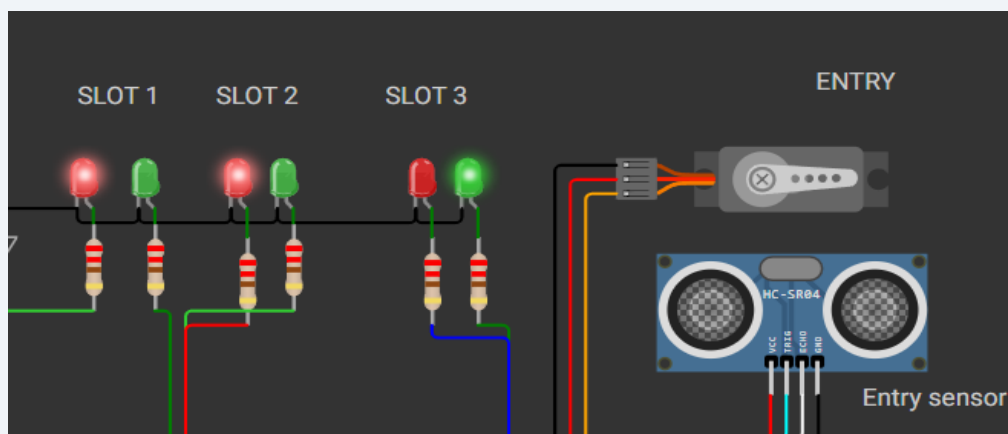
displayMsg("Exit Detected", "Scan RFID Card");
String uid = waitForRFID();
int slot = findSlot(uid);
if (slot == -1 || !slotOccupied[slot]) {
    displayMsg("No Active", "Session"); return;
}
float minutes = (millis() - entryTime[slot]) / 60000.0;
float bill = minutes * RATE_PER_MIN;
slotOccupied[slot] = false;
digitalWrite(greenPins[slot], HIGH);
digitalWrite(redPins[slot], LOW);
openGate(exitServo, 90);
displayBill(minutes, bill);
delay(5000); openGate(exitServo, 0);
}

```

7. Project Photographs



Figure 2: OLED Status Display



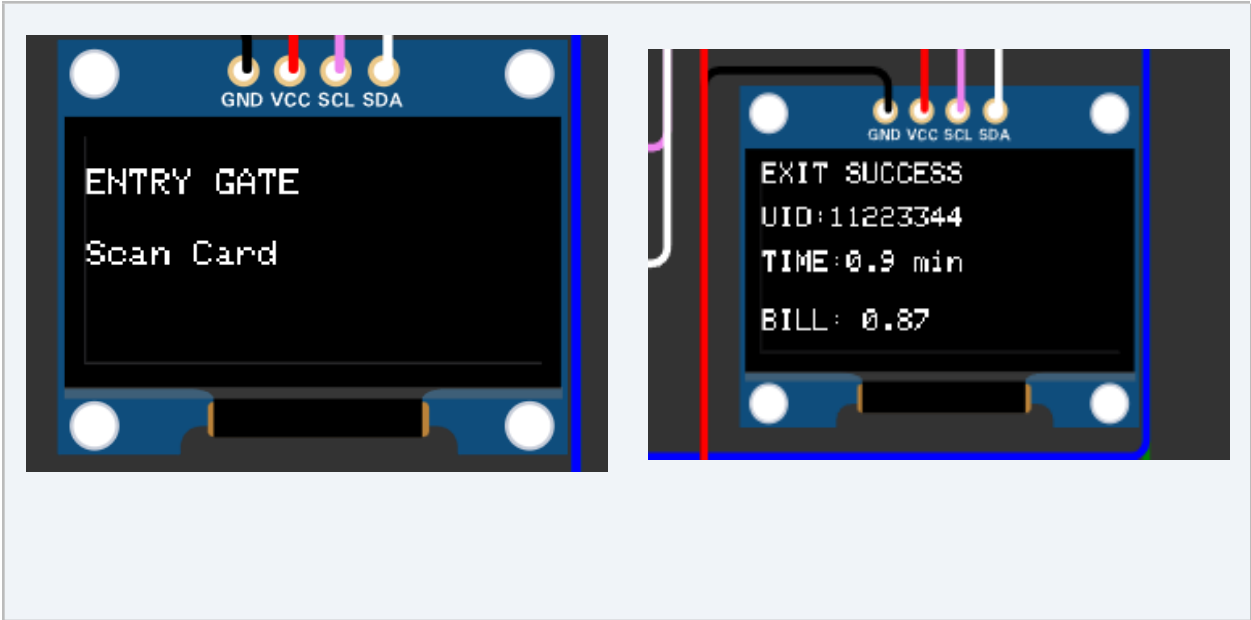
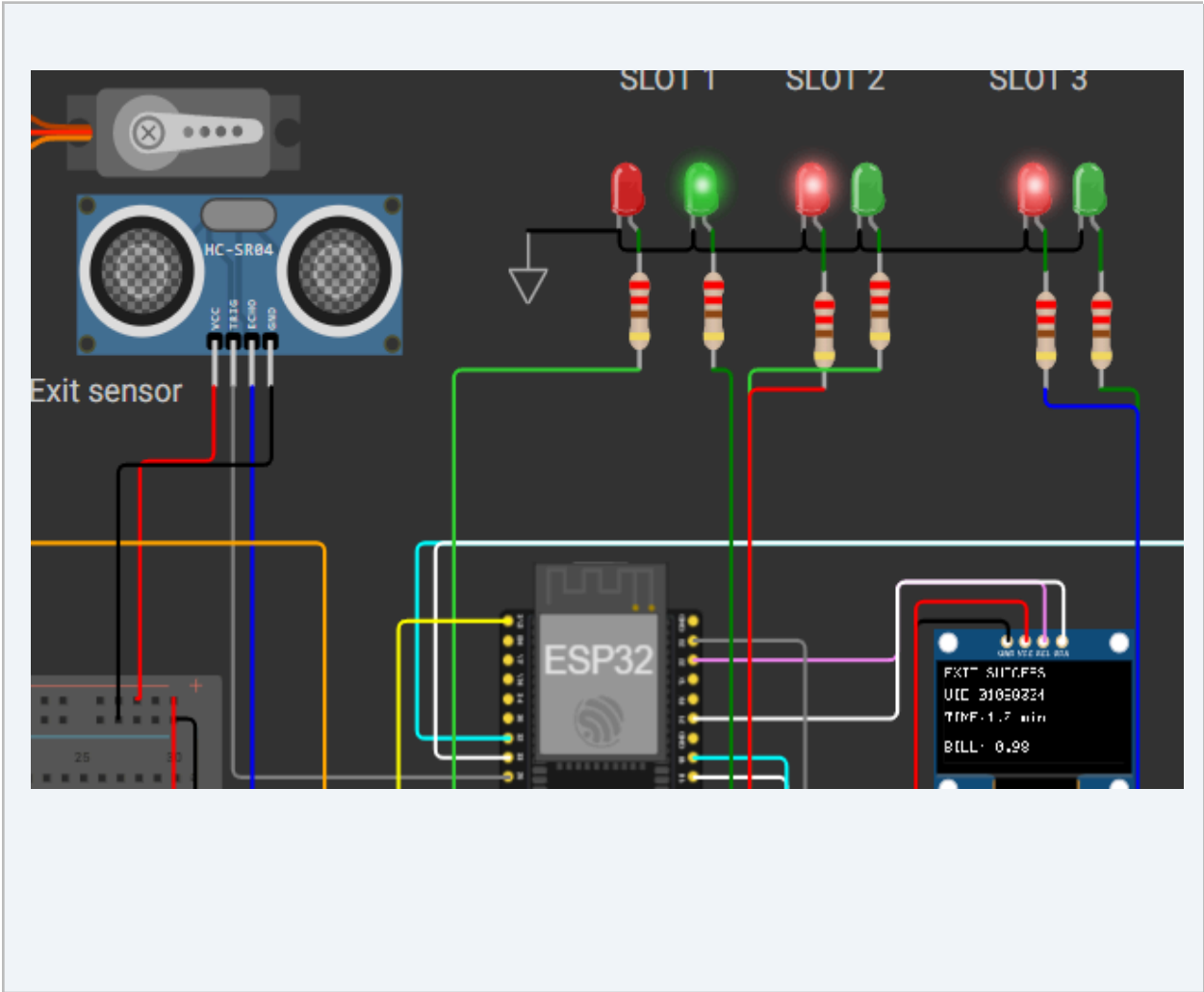


Figure 3: Entry Gate Operation



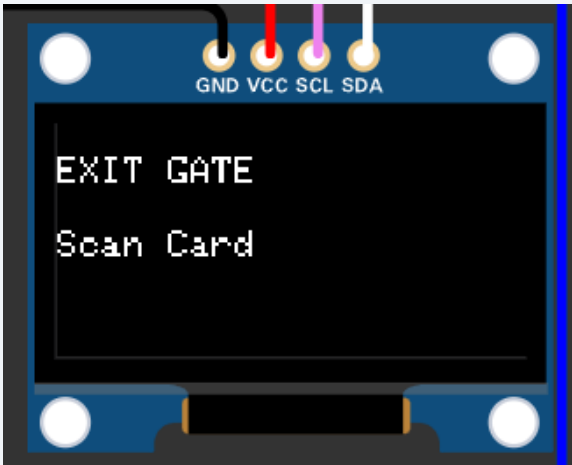


Figure 4: Exit Gate and Billing Display

8. Results and Discussion

8.1 System Performance

Test Parameter	Result	Observation
RFID read range	~3–5 cm	Reliable within specified range
Ultrasonic detection	< 15 cm threshold	Accurate vehicle detection
Gate open latency	< 1 second	Smooth servo response
OLED update speed	< 100 ms	Near-instantaneous display
Billing accuracy	Tested at 1 min, 5 min, 10 min	Correct to two decimal places
Multi-slot independence	All 3 slots tested simultaneously	No cross-slot conflicts

8.2 Limitations

- The current system supports only three pre-registered RFID cards; scalability requires an array expansion or external database.
- Parking duration is tracked using millis(), which resets on ESP32 reboot.
- The system does not incorporate wireless connectivity (Wi-Fi/cloud) for remote monitoring.

- Physical gate arms are simulated with servo motors and require mechanical reinforcement for real-world deployment.

9. Conclusion

This project successfully demonstrates an automated smart parking management system built around the ESP32 microcontroller. By integrating RFID-based vehicle identification, ultrasonic vehicle detection, servo-controlled gates, LED slot indicators, and an OLED billing interface, the system addresses common inefficiencies found in conventional manual parking management.

The key advantages achieved include:

- Contactless and secure vehicle identification through RFID technology, reducing human error and eliminating the need for paper tickets.
- Real-time slot occupancy monitoring via LED indicators and OLED display, enabling drivers and operators to view parking availability instantly.
- Automated gate control, improving traffic flow and reducing queue times at entry and exit points.
- Transparent and automatic billing based on actual parking duration, eliminating disputes and manual fare collection.
- Low-cost implementation using widely available embedded components, making it suitable for small-scale parking facilities, schools, hospitals, and commercial buildings.

Future enhancements could include Wi-Fi/MQTT integration for cloud-based monitoring, a mobile application for advance slot reservation, camera-based license plate recognition, and solar-powered operation for sustainability. The project demonstrates that intelligent infrastructure solutions can be implemented cost-effectively using readily available microcontroller technology.
